Table of Contents

Merseyside Area Group ESP32 Devkit 1 Can USB Build Guide....1
Hardware Setup Steps....1
Downloading The Firmware....2
Uploading the Firmware....3
JMRI Connection Setup....3
Seeing Your DIY Nodes....4

Merseyside Area Group ESP32 Devkit 1 Can USB Build Guide

This guide will walk you through the steps to build a PC to Can bus Grid Connect adapter for OpenLCB. We use this to be able to configure our nodes through JMRI (Java Model Railroad Interface).

The device requires you to have the following items, to be able to complete the build of the device.

- ESP32 Devkit 1 30 pin development board. AliExpress link
- ESP32 Devkit 1 30 pin break out board. AliExpress link
- SN65HVD230 Can Transceiver module. AliExpress link
- A suitable USB Data cable.
- 4 100mm female to female Dupont connectors
- A PC with the Arduino IDE installed.
- A PC with JMRI and Java installed.


ESP32 Devkit 1



ESP32 Shield

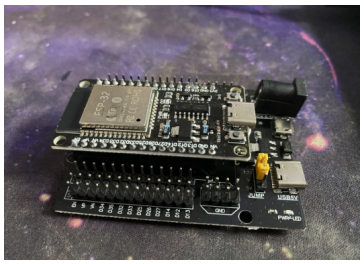


SN65HVD230

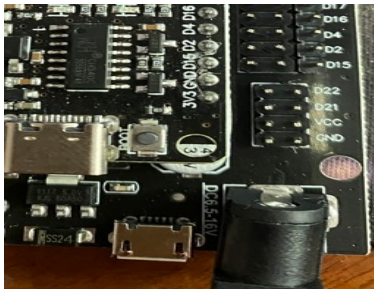
This guide will only cover the building and installing the firmware required for this device to work.

Hardware Setup Steps

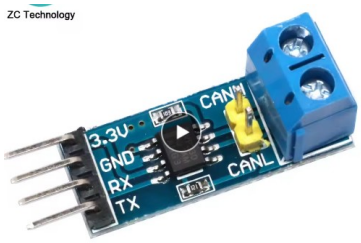
- Install the ESP32 into the breakout shield, ensuring that the 30 pins are lined up correctly, and that the USB is in the same direction as those on the shield.
- You may find that the build quality requires careful bending of the header pins on the shield or the ESP32 to get them to engage nicely.
- Ensure the small jumper on the shield is set to supply 3.3 volts and not 5 volts, this applies to the V rail output voltage.


ESP32 & Shield

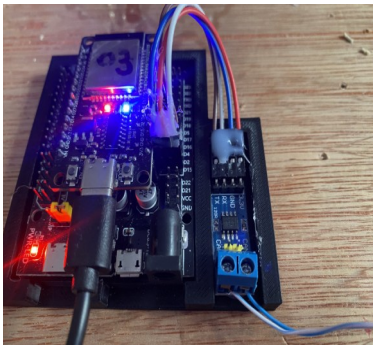
- Locate on the shield the following two pins D15 and D2. These will be on the right-hand side. When the USB connections are facing you.
- The are 3 pins associated with each D pin on the shield. At the end away from the USB you will see the labels for these pins. G (Ground) V (3.3 volts) S (Data / Signal)
- For the Can Transceiver we need to make the following connections using 4 of the 100mm female to female Dupont connectors.
  The SN65HVD230 has four connections at one end which are marked with. 3.3V, GND, RX, TX.
- Take note that between text TX and the CAN L you will see a small surface mounted resistor marked 121. This is a 120 Ohm termination resistor. This may need to be removed depending on were you place this device on your Can bus network.
- You need two 120 Ohm termination resistors at either end of your Can Bus network to ensure reliable transfer of Can messages on your network.
- I recommend that you separate the 4 Dupont connectors to make it easy to route the cables.
- Take one of the cables and connect GND on the shield to GND on the Transceiver module.
- Next connect the V on the shield to the 3.3V on the Transceiver module.
- Next connect D15 on the shield to RX on the Transceiver module.
- Finally connect D2 on the shield to TX on the Transceiver module.
- Once tested I tend to use hot glue to hold these connectors in place.



D15 & D2 Can Pins

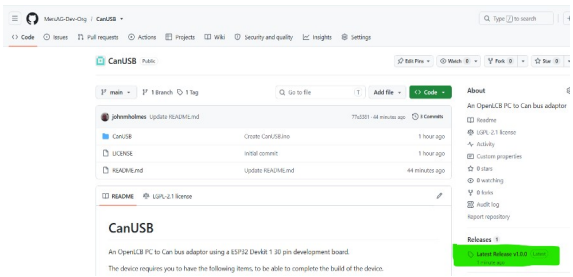


SN65HVD230 Module

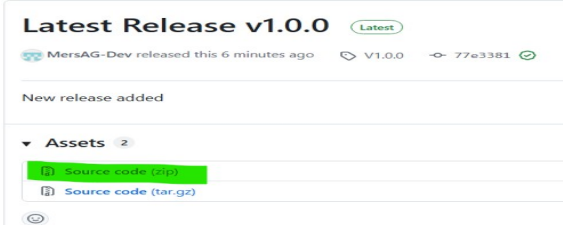

Completed Build

Downloading The Firmware.

- The firmware is held in our GitHub repository which you can download from this Link. You need to look for the Release section to get the Zip file onto your PC. You will notice I have highlighted it in green on the right-hand of the repository. Once you click on the link another window will open which offers two types of Zip files. I use Windows machine so it looks like this for me.
- Choose the Zip file type you need to use. Once you click on the type it may offer you a location to save the file on your PC. In my case it defaults to my download folder. Which I tend to use for this type of activity.

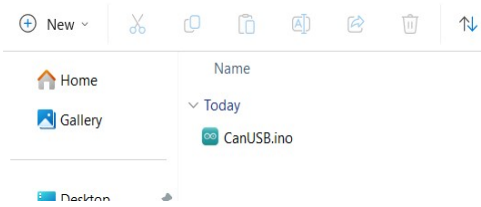


Release file location



Zip file choices

- Once you have the file downloaded and gone to the folder you choose you need to Extract the file to be able to get to the Arduino .INO file.
- Open the folder and navigate until you get into the folder with the CanUSB file as shown. It may take 3 clicks to get to the correct folder.
- Next double click on the file to open it in the Arduino IDE. In my case I am use the latest stable release. 2.3.10



Locate the CanUSB.ino file

Uploading the Firmware

This software needs you to have the ESP32 Board manger by Espressif. It has been tested with version 3.3.12 installed. You may need to have the correct UART driver installed. There are two common chips used on the development boards.
CH340 or the CP210x

CH340 driver is available from here
CP210x Driver is available from here



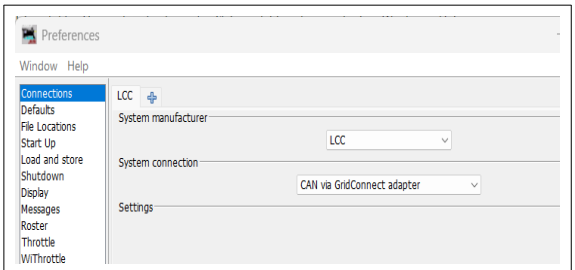
Set the correct ESP32 Device

- Set the correct Dev board type to ESP32 DEV Module.
- Set the correct COM port number
- Click on the Compile and Upload icon. The software will take a short while to upload and then the firmware is ready to use.
- You will notice if your Dev board has an LED on D2 it will illuminate.

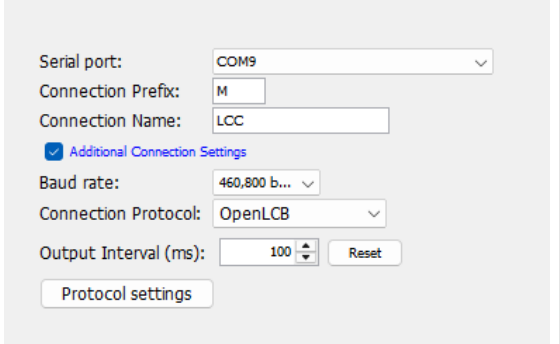
JMRI Connection Setup.

As our configuration tool is JMRI we need to setup a connection for JMRI to find the device. First open up Panel Pro. You could use LCC Pro if you wish. But I find Panel Pro does everything I need it to do.

- Open the Preferences window. On a Windows machine this is located under the Edit Tab.
- Click on the + icon to add a new connection.
- Choose LCC for System Manufacturer.
- Choose CAN via GridConnection Adapter.
- Set the Serial port to suit your connection. In my case COM9. Also when uploading the sketch in the Arduino IDE.
- Tick the Additional Connection Settings.
- Change the Baud rate to 460,800.

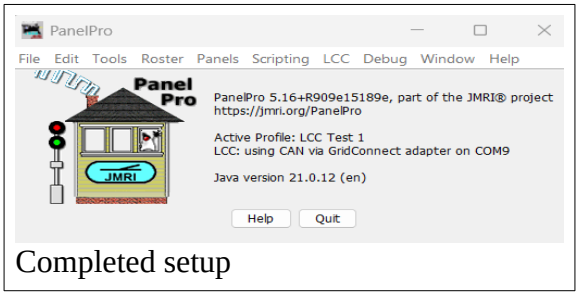


Setup the System settings.



Final setting in JMRI

- Finally you need to click on the save button. On Windows this is on the bottom left-had part of the preferences window.
- JMRI needs to restart.
- After restarting you will see your new connection as shown here.
- You are now ready to configure your DIY ESP32 nodes.

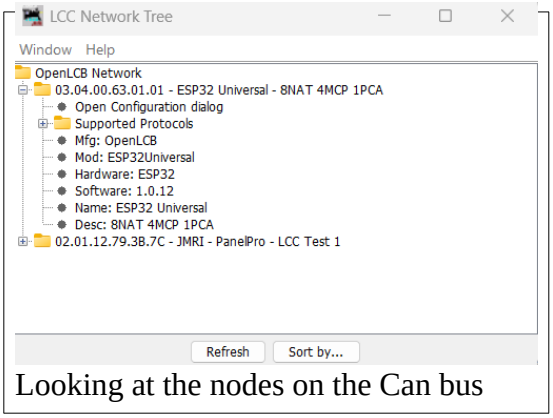


Completed setup

Seeing Your DIY Nodes.

To see your nodes you need to click on the LCC Tab on the splash screen and the choose. Configure Nodes. This will open the LCC Network Tree window.

- Here you can see two nodes. The 02.02.12** is ESP32 CanUSB.
- The other is in this case the MersAG Universal node.
- Clicking on Open Configuration Dialog will open the CDI for our node.
- More details for the node is available from our main repository here



Looking at the nodes on the Can bus